AWS PostgreSQL CloudOps

Complete project guide in simple words · Lakshmi Nair · github.com/Lechulakshmi821 · Day 1 to 3 Nov 2026 · work in progress

1. The big picture (in simple words)

> **Analogy:** Think of a small hotel. The **VPC** is the plot of land with a fence. The **public subnet** is the reception, where guests can walk in. The **private subnets** are the locked back rooms. The **database** is the safe in the back room: nobody from the street can reach it, only the receptionist (the test client) has the key. **CloudFormation** is the building plan: with one command AWS builds the whole hotel, and with another command it takes it down again.

What I built: a PostgreSQL database on AWS that is secured, will be monitored, backed up and restore-tested, and everything is written as code so it can be rebuilt any time.

```
Me (AWS CLI on my laptop)
   |  CloudFormation deploys the templates
   v
VPC 10.0.0.0/16, region Frankfurt (eu-central-1)
 +-- Public subnet : test client (EC2). I reach it with Session Manager (no SSH keys)
 |        |  port 5432, only allowed from the client's security group
 |        v
 +-- Private subnets A and B : RDS PostgreSQL 17 (no public access, encrypted, SSL required)

Secrets Manager : stores the generated database password
CloudWatch + SNS: alarms and email alerts (template ready, deploy next)
```

What I want to show employers: I can run a database in the cloud like I ran Oracle at Wipro: build it repeatably, secure it, watch it, and recover it.

2. What we did, from Day 1 until now

When	What
Day 1 24 Oct	Created the public GitHub repo `aws-postgres-cloudops` and defined the plan: network, RDS, monitoring, backups, restore test, cleanup. (From my notes; add details if you did more.)
Day 2 26 Oct	AWS account with **MFA**, an **IAM admin user** (instead of root), a **budget alert**, and the first CloudFormation **network stack** deployed.
Day 3 3 Nov	1. Installed the AWS CLI on Windows and set up access for my IAM user (`lakshmi-admin`); checked who I am with `aws sts get-caller-identity`. 2. Deployed the network again with export names (so other stacks can use the VPC and subnets); checked 6 exports. 3. Deployed the test client (EC2 with Session Manager) and the RDS PostgreSQL database. 4. Connected from the client with `psql` over TLS 1.3. `SHOW ssl` said `on`. 5. Created table `customers` with 3 rows (Alice, Bob, Carla) as test data for the restore test. 6. **Security test:** connecting with `sslmode=disable` was refused, which proves SSL is enforced. 7. Wrote the monitoring template (alarms, dashboard, email alerts) and checked it with `cfn-lint`.
Next	Deploy monitoring and test an alert; snapshot; delete a row on purpose; restore into a new database; measure the time; read-only database user; CloudTrail; screenshots; README; delete everything.

3. The AWS parts explained simply

Part	What it is, in simple words

IAM user / role	IAM is AWS's "who may do what". A **user** is a person (me); a **role** is a set of permissions a machine can use (the test client). Root account = the owner key; I don't use it daily.
MFA	Second login step (code from an app). Protects the account if the password leaks.
Budget alert	An email if AWS costs go above a limit I set.
VPC	My own private network in AWS (the fenced plot).
Subnet	A section of the VPC. **Public** = has a route to the internet. **Private** = no route, unreachable from outside.
Internet gateway	The front door of the VPC to the internet. Only the public subnet's route table uses it.
Security group	A firewall on a resource. Rule used here: the database accepts port 5432 only from the client's security group.
EC2	A virtual Linux server. Mine (t3.micro) is the test client.
Session Manager	Lets me open a terminal on the server through AWS, with no open SSH port and no key files.
RDS	AWS runs the database for me (installation, patching, backups). I still manage users, tuning and data. Here: PostgreSQL 17.6, db.t3.micro, 20 GB.
Secrets Manager	A safe for passwords. CloudFormation generates the database password and stores it there. Never in code or GitHub.
CloudFormation	Infrastructure as code. A **template** (YAML) describes resources; a **stack** is one deployed copy. Delete the stack and everything it created is removed.
CloudWatch	Metrics (CPU, storage, connections), logs, alarms and dashboards.
SNS	Sends notifications, here an email when an alarm fires.
CloudTrail (planned)	A record of who did what in the AWS account (an audit log).

4. What the YAML files contain and how they work

Every template has the same 3 blocks:

- `Parameters`: values I give at deploy time (my IP address, my email).
- `Resources`: the things to create (VPC, server, database...). This is the main part.
- `Outputs`: values shown after deploy (instance ID, database address). With `Export`, another stack can use them via `ImportValue`.

Handy helpers: `!Ref` = "use that other resource or parameter"; `!Sub` = text with variables; `!GetAtt` = read a property (like the database address); `!Select / !GetAZs` = pick an availability zone.

network.yaml

Creates the VPC (10.0.0.0/16), one public subnet (10.0.0.0/24) and two private subnets (10.0.10.0/24 and 10.0.11.0/24) in two zones, the internet gateway, the public route table and two security groups: **client** (SSH only from my IP) and **database** (port 5432 only from the client group). Exports six values.

Why it matters: the database sits in private subnets, and only the client may talk to it.

client.yaml

Creates the EC2 server (Amazon Linux 2023, t3.micro) in the public subnet, plus an IAM role with two rights: Systems Manager (so Session Manager works) and read access to **only** the database secret. The AMI (server image) is found automatically through an AWS parameter.

Why it matters: least privilege, and no SSH keys to lose.

database.yaml

Creates a DB subnet group (the two private subnets), a parameter group (`rds.force_ssl=1`, log connections, log disconnections, log queries slower than 1 second), a secret with a generated 24-character password, and the

RDS instance: not public, storage encrypted, logs sent to CloudWatch, 1 day of automatic backups. The username and password are read from Secrets Manager with `{{resolve:secretsmanager:...}}`, so they never appear in the file.

Why it matters: secure by default and no password in Git.

monitoring.yaml (ready, not deployed yet)

Creates an SNS topic with my email, four alarms (CPU over 80%, free storage under 5 GB, more than 50 connections, free memory under 100 MB) and a CloudWatch dashboard with four graphs.

Why it matters: I find problems before the users do.

How it all works together

1. `network.yaml` builds the network and exports its IDs.
2. `client.yaml` and `database.yaml` import those IDs, so they land in the right subnets with the right security groups.
3. I log in to the client through Session Manager, fetch the password from Secrets Manager and connect to the database with `psql`.
4. `monitoring.yaml` watches the database; deleting the stacks (in reverse order) removes everything.

5. The commands and what each one means

Command	Meaning
`aws --version`	Shows that the AWS CLI is installed.
`aws configure`	Saves my access key, region (eu-central-1) and output format on my laptop.
`aws sts get-caller-identity`	"Who am I?" Shows the user I am logged in as. Must be my IAM user, not root.
`(Invoke-WebRequest ... checkip.amazonaws.com).Content`	PowerShell: asks a website for my public IP address, needed to allow only my IP in the network stack.
`aws cloudformation deploy`	Creates or updates a stack from a template. Parts: `--region` (where), `--template-file` (which YAML), `--stack-name` (name of the deployed copy), `--parameter-overrides` (give parameters, e.g. MyIp), `--capabilities CAPABILITY_IAM` (allow it to create IAM roles).
`aws cloudformation list-exports --query "Exports[].Name"`	Lists the exported values; `--query` filters the answer.
`aws cloudformation describe-stacks --query ... --output text`	Reads a stack's details, here the server ID or database address from the Outputs.
`Select-String -Path file -Pattern "Export"`	PowerShell search inside a file (like grep). Used to check my YAML had exports.
`aws ssm start-session --target i-...`	Opens a terminal on the server through Session Manager. `i-037ace...` is the server's ID.
`sudo dnf install -y postgresql17`	Installs the PostgreSQL client (`psql`) on the server. `sudo` = as administrator, `-y` = answer yes.
`export AWS_DEFAULT_REGION=eu-central-1`	Tells the AWS CLI on the server which region to use.
`export PGPASSWORD=$(aws secretsmanager get-secret-value ... | python3 -c ...)`	Reads the secret, picks the "password" field and stores it in a variable that `psql` uses, so I never see or type the password.
`psql "host=... dbname=appdb user=dbadmin sslmode=require"`	Connects to the database. `sslmode=require` = encrypted only; with `disable` the server refuses (my SSL test).

`CREATE TABLE` / `INSERT` / `SELECT`	SQL: make a table, add rows, read them back.
`SHOW ssl;`	Shows if the connection is encrypted (`on`).
`\q` and `exit`	Leave `psql`, then leave the server session.
`aws cloudformation delete-stack` + `wait stack-delete-complete`	Deletes a stack and waits until it is gone. Order: database, client, network.
`aws secretsmanager delete-secret --force-delete-without-recovery`	Removes the secret now, so its name can be used again.
`git add .` / `git commit -m "..."` / `git push`	Save the files in Git, with a message, and upload them to GitHub.

6. Interview questions and simple answers

About the project

Tell me about this project.

I built a PostgreSQL platform on AWS with CloudFormation: a VPC with public and private subnets, an RDS PostgreSQL database in the private subnets, a test client reached with Session Manager, the password in Secrets Manager, and monitoring and backup/restore testing as next steps. I came from Oracle DBA work, so I wanted to show the same duties in the cloud.

Why CloudFormation and not the console?

It is repeatable and reviewable. I can delete and rebuild everything in minutes and keep the history in Git.

What went wrong and what did you learn?

My first network template had no export names, so the other stacks could not find the VPC. I added exports and checked them with `list-exports`. I also learned to check templates with cfn-lint and to delete the secret during cleanup.

What would you improve?

Multi-AZ, a read replica, CloudTrail, pgaudit, a CI pipeline that runs cfn-lint, patching with Systems Manager, and Terraform as a comparison.

Security

How is the database protected?

Private subnets and no public access; port 5432 only from the client's security group; encryption at rest; SSL enforced (I tested it); password generated in Secrets Manager.

Why Session Manager instead of SSH?

No open inbound port and no keys to lose; access is controlled by IAM and can be logged.

What is least privilege? Where did you use it?

Give only the rights needed. The client's role can read only one secret, and the database accepts only the client's security group.

Why not use the root account?

Root cannot be restricted. I use an IAM user with MFA.

Security group vs network ACL?

A security group is a stateful firewall on the resource; a network ACL is a stateless firewall on the subnet. I used security groups.

Networking and AWS basics

Public vs private subnet?

A public subnet has a route to the internet gateway; a private one does not.

Why two private subnets?

An RDS subnet group needs two availability zones. It also allows Multi-AZ later.

Why no NAT gateway?

It costs money every hour and the database does not need internet access.

What is a CloudFormation stack, and what does delete do?

A stack is one deployed copy of a template. Deleting it removes the resources it created.

What are Parameters, Resources, Outputs?

Inputs at deploy time, the things to create, and values shown after deploy (also exported to other stacks).

Operations: monitoring, backup, recovery

What do you monitor on a database?

CPU, free storage, freeable memory, connections, slow queries and failed logins in the logs, later replication lag and backup status.

What RDS backup options exist?

Automated backups with point-in-time recovery (1 to 35 days) and manual snapshots that stay until I delete them.

How does an RDS restore work?

It always creates a new instance from a snapshot or a point in time; it never overwrites the old one. In my test I will delete a row, restore, check that the rows are back and note the time.

What are RPO and RTO?

RPO = how much data loss is acceptable (point-in-time recovery: about 5 minutes). RTO = how long recovery may take; my restore test measures it.

How does this compare to your Oracle work?

Same duties: backup and recovery, monitoring, patching, access control and on-call. RMAN backups relate to RDS snapshots and PITR, Data Guard and RAC relate to Multi-AZ and read replicas, AWR/ADDM relate to Performance Insights and CloudWatch.

How do you control cost?

Budget alert, smallest sizes, no NAT gateway, and deleting the stacks after each session.

Do you have Kubernetes or Terraform experience?

Not yet. This project uses CloudFormation. I understand the idea (infrastructure as code) and am learning step by step. (Be honest about this.)

7. What comes next

1. Deploy `monitoring.yaml`, confirm the email, test an alarm.
2. Take a manual snapshot and set automated backups to 7 days.
3. Restore test: delete a row, restore into a new instance, check the data, record the time.
4. Read-only database user (least privilege) and CloudTrail.
5. Screenshots in `docs/`, update README and this guide, push to GitHub.
6. Delete all stacks.

> Never put access keys, passwords, your account number or your IP address into GitHub or screenshots you share.